

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Mai Đức Minh Huy
Chung Tín Đạt

Image Segmentation Based on Visual
Prompt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 01/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Mai Đức Minh Huy - 23122008

Chung Tín Đạt - 23122024

Image Segmentation Based on Visual
Prompt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
PGS.TS. LÝ QUỐC NGỌC

Tp. Hồ Chí Minh, tháng 01/2026

Contents

1	Introduction	1
1.1	General Background	1
1.2	Motivation	3
1.3	Problem Statement	4
1.4	Contribution	6
2	Related Works	9
2.1	Traditional Methods	9
2.1.1	K-Means Clustering	9
2.1.2	Mean Shift Clustering	11
2.1.3	Binary Thresholding	13
2.2	Modern Methods	14
2.2.1	Fully Convolutional Networks (FCNs)	14
2.2.2	U-Net	15
2.2.3	Mask R-CNN	17
2.2.4	DeepLab	19
2.2.5	Transformer-Based Segmentation	20
2.2.6	Recurrent Neural Networks for Segmentation	22
3	Methodology	25
3.1	The SA-1B Data Engine — The Flywheel of Generalization	25
3.1.1	Paradigm Shift: From Manual Annotation to Model-in-the-Loop	25

3.1.2	Stage 1: Assisted-Manual Annotation (4.3M Masks, 120K Images)	26
3.1.3	Stage 2: Semi-Automatic Annotation (5.9M Additional Masks, 180K Images)	27
3.1.4	Stage 3: Fully Automatic Annotation (1.1B Masks, 11M Images)	28
4	Cài đặt và thử nghiệm	34
5	Kết luận và hướng phát triển	35
5.1	Conclusion	35
5.2	Discussion	35
5.2.1	Limitations	35
5.2.2	Future Work	35

Chapter 1

Introduction

1.1 General Background

Image segmentation is a fundamental task in computer vision that involves partitioning a digital image into multiple segments or sets of pixels, often corresponding to different objects or parts of objects. The goal is to simplify or change the representation of an image into something that is more meaningful and easier to analyze. The history of this field can be broadly divided into three distinct eras.

The Pre-Deep Learning Era (c. 1970s–2000s): Before the widespread adoption of deep learning, segmentation methods were primarily based on handcrafted features and mathematical models. These techniques required significant domain knowledge and focused on pixel-level properties such as intensity, color, texture, and edges. Prominent methods from this period include:

- **Canny Edge Detection:** Proposed by John Canny in 1986, this method became a standard for detecting a wide range of edges in images through a multi-stage algorithm [[canny1986computational](#)].
- **Active Contour Models (Snakes):** Introduced by Kass et al. in 1988, these deformable models were particularly effective for segmenting objects with irregular shapes, finding extensive use in med-

ical imaging [kass1988snakes].

- **K-Means Clustering:** This unsupervised algorithm, formalized by MacQueen in 1967, partitions pixels into clusters based on feature similarity, serving as a foundational clustering-based segmentation technique [macqueen1967some].

These methods, while foundational, relied on heuristics and low-level features, often struggling with complex scenes and semantic understanding.

The Deep Learning Era (c. 2010s): The advent of deep learning, particularly Convolutional Neural Networks (CNNs), revolutionized image segmentation. These models could automatically learn hierarchical features from data, eliminating the need for manual feature engineering. Key milestones include:

- **Fully Convolutional Networks (FCN):** Long et al. (2015) introduced FCNs, which replaced fully connected layers in CNNs with convolutional layers, enabling end-to-end training for dense, pixel-wise prediction. The use of skip connections allowed the combination of deep, semantic features with shallow, appearance features [long2015fully].
- **U-Net:** Proposed by Ronneberger et al. (2015), U-Net features a symmetric encoder-decoder architecture with extensive skip connections. This design proved highly effective for biomedical image segmentation, achieving remarkable performance with very few training images [ronneberger2015u].
- **Mask R-CNN:** He et al. (2017) extended the Faster R-CNN object detection framework by adding a parallel branch for predicting segmentation masks. Mask R-CNN became a dominant framework for instance segmentation and a versatile backbone for many downstream tasks [he2017mask].

Modern Research Directions: Current research continues to build upon these deep learning foundations, exploring new architectures and learning paradigms for greater accuracy, efficiency, and generalization.

- **Transformer-Based Architectures:** Vision Transformers (ViTs) and self-attention mechanisms have been adapted for segmentation, leading to models like TransUNet (2021) that combine the strengths of Transformers and U-Net [chen2021transunet].
- **Foundation Models:** A recent paradigm shift involves building large-scale, pre-trained models that can be adapted to various tasks with minimal fine-tuning. The Segment Anything Model (SAM) from Meta AI (2023) exemplifies this, offering prompt-based, zero-shot segmentation for images [kirillov2023segment]. Its successor, SAM 2 (2024), extends these capabilities to video [ravi2024sam].
- **Self-Supervised Learning:** Techniques like contrastive learning (e.g., SimCLR) are being explored to reduce reliance on large labeled datasets, enabling models to learn powerful representations from unlabeled data.

1.2 Motivation

The drive to advance image segmentation stems from both scientific inquiry and practical necessity.

Scientifically, segmentation addresses a fundamental question in perception: while image classification tells us *what* is in an image, segmentation answers *where* it is, down to the pixel level. This granular understanding is crucial for emulating the human ability to parse a scene into distinct objects and regions, forming the basis for true visual comprehension.

Practically, segmentation is an enabling technology for a multitude of real-world applications:

- **Autonomous Driving:** Segmenting lanes, pedestrians, vehicles, and traffic signs is critical for safe navigation.
- **Medical Imaging:** Precise segmentation of tumors, organs, and other anatomical structures is vital for diagnosis, treatment planning, and surgical guidance.
- **Real-World Data Complexity:** Segmentation provides a robust way to handle complex scenes with occluded or overlapping objects.
- **Downstream Vision Tasks:** It serves as a foundational step for numerous other tasks, including object detection, tracking, and detailed semantic scene understanding. Without accurate segmentation, the performance of these downstream applications would be severely limited.

The motivation behind the **Segment Anything Model (SAM)** represents a culmination of these goals. As stated by its creators, “In this work, our goal is to build a foundation model for image segmentation” [kirillov2023segment]. The aim was not to create another specialized model but a generalist one. They sought “to develop a promptable model and pre-train it on a broad dataset using a task that enables powerful generalization. With this model, we aim to solve a range of downstream segmentation problems on new data distributions using prompt engineering” [kirillov2023segment]. This ambition marks a shift towards creating a single, powerful tool that can handle any segmentation request on any image, democratizing the technology and accelerating research and application development.

1.3 Problem Statement

Image segmentation is the task of grouping pixels that belong to a common semantic object and separating them from other objects and the

background. A "semantic object" refers to an entity that is meaningful and recognizable within a given context, such as a car, a person, a tree, or even amorphous regions like "sky" or "road".

The core principle of segmentation is guided by two criteria: pixels within the same group should be as similar as possible, while pixels belonging to different groups should be as distinct as possible. This "similarity" can be defined by various attributes, including color, intensity, texture, or learned semantic features. In essence, the task simulates the human cognitive ability to transform raw, unstructured visual data into a structured representation of distinct entities.

For a modern promptable segmentation system like SAM, the problem can be formally defined by its inputs and outputs.

- **Input:** An image and a "visual prompt" that specifies the object of interest. This prompt can take various forms, such as a point (or multiple points) on the object, a bounding box enclosing it, or even a rough mask.
- **Output:** One or more segmentation masks for the object(s) indicated by the prompt, along with a confidence score for each mask. The ability to output multiple masks is crucial for handling ambiguity (e.g., a single point on a shirt could refer to the shirt or the person wearing it).

The general framework for such a system, as shown in Fig. 1.1, typically consists of two main stages: a powerful visual encoder that processes the input image to create a high-dimensional feature representation, and a lightweight decoder that takes this representation along with an encoded prompt to rapidly generate the final mask.

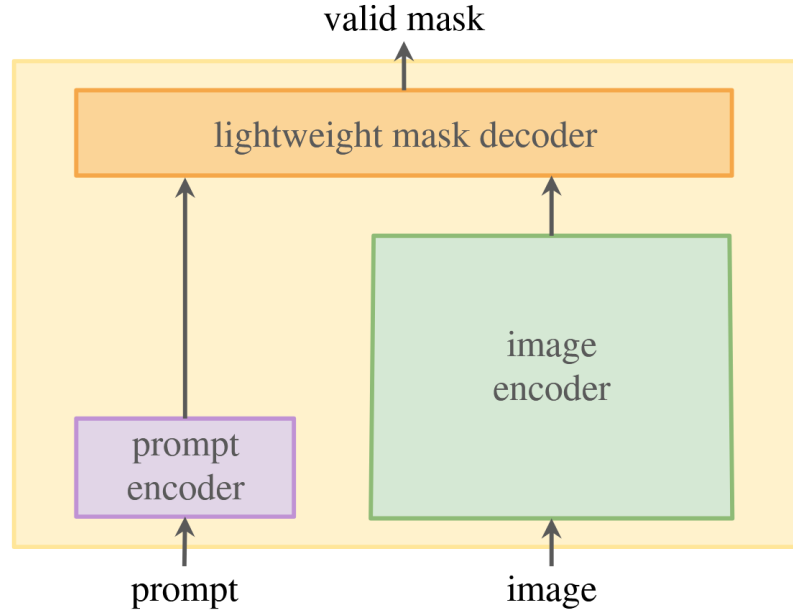


Figure 1.1: A general framework for a promptable segmentation model. An image encoder computes a feature embedding for the input image. A prompt encoder embeds the user’s prompt (e.g., points, boxes). A lightweight mask decoder combines these embeddings to predict segmentation masks in real-time.



Figure 1.3: Diagram for Input/ Output here + Framework etc etc

1.4 Contribution

The work on the Segment Anything Model (SAM) introduced several pivotal contributions to the field. As summarized by the authors, “Our principal contributions are a new task (promptable segmentation), foundational model for that new task (SAM), and dataset (SA-1B) that make this leap possible” [kirillov2023segment].

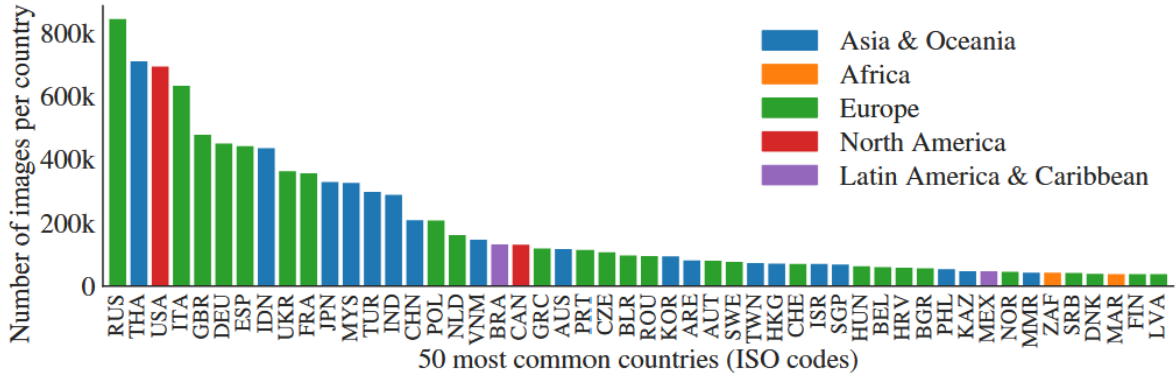


Figure 1.2: Estimated geographic distribution of SA-1B images. The map shows that most countries are represented with over 1000 images, making it one of the most geographically diverse datasets for computer vision. (Source: [kirillov2023segment])

A key contribution is the **SA-1B dataset**, which remains the largest segmentation dataset to date. Its key features include:

- **Scale:** It contains over 11 million high-resolution images and 1.1 billion high-quality segmentation masks.
- **Privacy-First:** All images are privacy-respecting, with automated blurring of faces and license plates.
- **Diversity:** The dataset is geographically diverse, with a significant number of images from countries around the world, as illustrated in Fig. 1.2. This helps mitigate geographical bias often present in large-scale datasets.

The contribution of this report is to provide a comprehensive survey and structured analysis of the field of image segmentation. We synthesize the historical evolution, explain the motivations driving modern research, and offer a detailed examination of the methodology behind foundation models like SAM, contextualizing their impact and outlining future directions.

Table 1.1: Comparison Table - Part 1: Methodology

Papers	Mechanics	Methodology		
		Stage 1	Stage 2	Stage 3
copy copy	More table copy ^a			

^aSample of a Table footnote.

Table 1.2: Comparison Table - Part 2: Performance and Analysis

Papers	Performance			Contribution	Limitation
	Data	Metric	Result		
copy					

^aSample of a Table footnote.

Chapter 2

Related Works

2.1 Traditional Methods

2.1.1 K-Means Clustering

K-Means is an unsupervised clustering algorithm used to segment regions of interest from the background [nameirakpam2015kmeanssubtractive]. It was first introduced by Hartigan in 1975, and its goal is to partition M points in an N -dimensional space into K clusters such that the within-cluster sum of squares is minimized [hartigan1979kmeans].

1. **Mechanics:** The algorithm iteratively assigns each pixel to the nearest cluster centroid and updates centroids based on current assignments. K-Means algorithm seeks a local optimum rather than a global one. Data points are divided into groups by assigning each of them to the nearest centroid. The goal is to find a solution such that moving any point from one cluster to another does not reduce the within-cluster sum of squares [hartigan1979kmeans].
2. **Methodology:** Let $p(x, y)$ be a pixel at position (x, y) to be cluster and c_k be the centroid of cluster k . Let C_k be the set of pixels assigned to cluster k , where (x, y) indexes the image pixels and $k \in \{1, 2, \dots, K\}$.

For each pixel, compute its distances to all cluster centroids:

$$D_{x,y} = \{ \|p_{x,y} - c_k\| \mid k \in \{1, 2, \dots, K\} \}$$

Assign the pixel to the cluster with the smallest distance:

$$n = \arg \min_k \|p_{x,y} - c_k\|,$$

$$C_n = C_n \cup \{p_{x,y}\}$$

After all pixels are assigned, update each centroid as the mean of the pixels belonging to it:

$$c_k = \frac{1}{|C_k|} \sum_{p_{x,y} \in C_k} p_{x,y}$$

Repeat the assignment and update steps until the centroids no longer change significantly or a maximum number of iterations is reached.

3. **Input:** An image represented by M pixels, each with N features (e.g., RGB color components or combined color-spatial features), forming an $M \times N$ matrix. The algorithm also takes K initial cluster centroids in the same N -dimensional feature space.

4. **Output:**

- A label map of size $M \times 1$ indicating the cluster each pixel belongs to.
- A vector of size $K \times 1$ containing the number of pixels in each cluster.
- A $K \times N$ matrix of the final cluster centroids.

5. **Algorithm:**

```

1      1. Initialize cluster centroids
2      2. For each pixel (x, y):
3          Compute its distance to all centroids
4          Assign it to the cluster with the smallest distance
5      3. For each cluster:
6          Recalculate the centroid as the mean of all pixels assigned to
it
7      4. Repeat steps 2 and 3 until convergence
8

```

2.1.2 Mean Shift Clustering

Mean Shift is a non-parametric clustering algorithm, which falls into unsupervised clustering category. Unlike K-Means, it does not require specifying the number of clusters in advance. Instead, it locates regions of high data density (modes) by iteratively shifting data points toward the nearest local maximum of a kernel density estimate [comaniciu2002mean].

1. **Mechanics:** The algorithm treats data points as samples from an underlying probability density function and seeks its local maxima. Each pixel is iteratively moved toward regions of higher density, determined by a kernel function, until convergence. Pixels converging to the same mode belong to the same cluster.
2. **Methodology:** Let $p_{x,y}$ denote the feature vector of the pixel at position (x, y) . The goal is to move $p_{x,y}$ iteratively toward the local density maximum according to:

$$m(p_{x,y}) = \frac{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right) p_j}{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right)}$$

where:

- S is the set of all data points (pixels),

- $K(\cdot)$ is a kernel function (commonly a Gaussian kernel),
- h is the bandwidth controlling the kernel width.

The mean shift vector is defined as:

$$\Delta p_{x,y} = m(p_{x,y}) - p_{x,y}$$

and the pixel is updated iteratively:

$$p_{x,y}^{(t+1)} = p_{x,y}^{(t)} + \Delta p_{x,y}$$

until convergence. After convergence, all pixels that converge to the same mode are assigned to the same cluster.

3. **Input:** An image represented by M pixels, each having an N -dimensional feature vector. The algorithm also requires a kernel bandwidth parameter h that defines the size of the neighborhood considered for density estimation.

4. **Output:**

- A segmented image in which each cluster corresponds to a region of similar features.
- The set of mode locations that represent cluster centers.

5. **Algorithm:**

```

1   1. For each pixel (x, y):
2       Initialize  $p_{x,y}$  with its feature vector
3   2. Repeat until convergence:
4       a. Compute the mean shift vector  $\Delta p_{x,y}$ 
5          using nearby pixels within bandwidth  $h$ 
6       b. Update  $p_{x,y} = p_{x,y} + \Delta p_{x,y}$ 
7   3. After convergence, group pixels whose final positions fall
      within the same mode region
8   4. Assign each group as one cluster
9
```


2.1.3 Binary Thresholding

Binary thresholding is used to separate the foreground (region of interest) from the background based on pixel intensity values. It is particularly effective when the objects and background have distinct intensity distributions.

1. **Mechanics:** Each pixel in the grayscale image is compared against a threshold value T . Pixels with intensity greater than or equal to T are classified as foreground, while the others are classified as background. This process converts a grayscale image into a binary one.
2. **Methodology:** For each pixel (x, y) in the image:
 - (a) Compare $I(x, y)$ with the threshold value T .
 - (b) Assign the binary value based on the comparison rule.

T can be predefined or determined adaptively using methods such as Otsu's algorithm, which selects the threshold that minimizes the intra-class variance between the foreground and background.

3. **Input:** A grayscale image, where (x, y) denotes the pixel coordinates, $I(x, y) \in [0, 255]$ represents its intensity, a threshold value T to determine the output pixel at (x, y) .
4. **Output:** A binary image B , where

$$B(x, y) = \begin{cases} 1, & \text{if } I(x, y) \geq T, \\ 0, & \text{otherwise.} \end{cases}$$

5. **Algorithm:**

```
1   For each pixel (x, y):  
2       If  $I(x, y) \geq T$ :  
3            $B(x, y) = 1$ 
```

```

4 Else :
5     B(x, y) = 0
6

```

2.2 Modern Methods

2.2.1 Fully Convolutional Networks (FCNs)

Fully Convolutional Networks, introduced by Long et al. in 2015, revolutionized semantic segmentation by adapting classification networks for dense prediction tasks [long2015fcn]. FCN replaced fully connected layers with convolutional layers, enabling end-to-end learning for pixel-wise segmentation.

1. **Mechanics:** FCN transforms classification networks (e.g., VGG, ResNet) into fully convolutional architectures by replacing dense layers with 1×1 convolutions. The network produces coarse feature maps through successive pooling, then upsamples them back to the original image resolution using transposed convolutions (deconvolution). Skip connections combine deep semantic features with shallow appearance features to recover spatial details lost during downsampling.
2. **Methodology:** Let $I \in \mathbb{R}^{H \times W \times 3}$ be the input image and f_θ represent the convolutional encoder producing feature maps at multiple scales. The encoder generates hierarchical features:

$$F_i = f_{\theta_i}(F_{i-1}), \quad F_0 = I$$

where F_i represents features at stage i with progressively reduced spatial dimensions.

Upsampling is performed via transposed convolutions g_ϕ :

$$U_i = g_{\phi_i}(U_{i-1})$$

Skip connections fuse features from encoder and decoder:

$$U_i = U_i + W_i \odot F_i$$

where W_i are learned weights and $\odot$ denotes element-wise multiplication.

The final prediction is obtained through a 1×1 convolution:

$$P = \text{softmax}(W_{\text{out}} * U_{\text{final}})$$

3. **Input:** An RGB image of arbitrary size $H \times W \times 3$.
4. **Output:** A probability map of size $H \times W \times C$, where C is the number of semantic classes. Each pixel is assigned to the class with highest probability.
5. **Algorithm:**

```

1   1. Pass image through convolutional encoder
2   2. Store feature maps at multiple scales
3   3. Upsample coarse feature maps via transposed convolutions
4   4. Add skip connections from encoder to decoder
5   5. Apply 1x1 convolution for class prediction
6   6. Output per-pixel class probabilities
7

```

2.2.2 U-Net

U-Net, proposed by Ronneberger et al. in 2015, features a symmetric encoder-decoder architecture with extensive skip connections [ronneberger2015un

Originally designed for biomedical image segmentation, U-Net achieves high accuracy even with limited training data.

1. **Mechanics:** U-Net consists of a contracting path (encoder) that captures context and an expansive path (decoder) that enables precise localization. The encoder applies repeated 3×3 convolutions and 2×2 max pooling for downsampling. The decoder uses 2×2 transposed convolutions for upsampling, concatenated with correspondingly cropped features from the encoder via skip connections. This architecture allows the network to propagate context information to higher resolution layers.
2. **Methodology:** The encoder path consists of repeated application of convolution blocks:

$$F_i^{\text{enc}} = \text{ReLU}(\text{BN}(W_i^{\text{enc}} * F_{i-1}^{\text{enc}}))$$

followed by max pooling for downsampling.

The decoder path upsamples and concatenates:

$$F_i^{\text{dec}} = [U_i, \text{crop}(F_{N-i}^{\text{enc}})]$$

where $[,]$ denotes channel-wise concatenation, U_i is the upsampled feature map, and crop ensures spatial alignment.

Each concatenated feature undergoes convolution:

$$F_i^{\text{dec}} = \text{ReLU}(\text{BN}(W_i^{\text{dec}} * F_i^{\text{dec}}))$$

The final segmentation map is generated by:

$$M = \sigma(W_{\text{final}} * F_{\text{final}}^{\text{dec}})$$

where σ is the sigmoid (for binary) or softmax (for multi-class) activation.

3. **Input:** An image of size $H \times W \times C$ (typically grayscale or RGB).
4. **Output:** A segmentation mask of size $H \times W \times K$, where K is the number of classes.
5. **Algorithm:**

```
1   1. Encoder: Apply convolutions and max pooling to downsample
2   2. Bottleneck: Process lowest resolution features
3   3. Decoder: Upsample via transposed convolutions
4   4. Concatenate encoder features via skip connections
5   5. Apply convolutions on concatenated features
6   6. Generate final segmentation mask
7
```

2.2.3 Mask R-CNN

Mask R-CNN, introduced by He et al. in 2017, extends Faster R-CNN for instance segmentation by adding a parallel branch for predicting segmentation masks [he2017mask]. This architecture excels at detecting and segmenting individual object instances.

1. **Mechanics:** Mask R-CNN operates in two stages. First, a Region Proposal Network (RPN) generates candidate object bounding boxes. Second, for each region of interest (RoI), the network performs three parallel tasks: class prediction, bounding box regression, and mask prediction. The key innovation is RoIAlign, which preserves spatial alignment during RoI pooling by using bilinear interpolation, crucial for pixel-accurate mask prediction.
2. **Methodology:** Let I be the input image and f_{backbone} be the feature extractor (e.g., ResNet-FPN).

Feature extraction:

$$F = f_{\text{backbone}}(I)$$

RPN generates proposals:

$$\{B_i, s_i\}_{i=1}^N = \text{RPN}(F)$$

where B_i are bounding boxes and s_i are objectness scores.

For each RoI, RoIAlign extracts aligned features:

$$F_{\text{RoI}} = \text{RoIAlign}(F, B_i)$$

Three parallel heads process F_{RoI} :

$$p_{\text{cls}} = \text{softmax}(W_{\text{cls}} \cdot F_{\text{RoI}}) \quad (\text{classification})$$

$$\Delta B = W_{\text{box}} \cdot F_{\text{RoI}} \quad (\text{box regression})$$

$$M = \sigma(W_{\text{mask}} * F_{\text{RoI}}) \quad (\text{mask prediction})$$

The loss function combines three terms:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}$$

3. **Input:** An RGB image of arbitrary size, typically resized to have a maximum dimension of 800-1024 pixels.
4. **Output:** For each detected instance: class label, confidence score, bounding box coordinates (x, y, w, h) , and a binary mask of size 28×28 (upsampled to the original bounding box size).

5. Algorithm:

1. Extract multi-scale features via backbone + FPN
2. Generate region proposals using RPN
3. For each RoI:

```

4      a. Apply RoIAlign for spatial alignment
5      b. Predict class probabilities
6      c. Refine bounding box coordinates
7      d. Generate instance segmentation mask
8      4. Apply Non-Maximum Suppression (NMS)
9      5. Output final detections with masks
10

```

2.2.4 DeepLab

DeepLab, developed by Chen et al., employs atrous (dilated) convolution and Atrous Spatial Pyramid Pooling (ASPP) to capture multi-scale contextual information without reducing spatial resolution [chen2017deeplab]. DeepLabv3+ further incorporates an encoder-decoder structure.

1. **Mechanics:** DeepLab uses atrous convolution to expand the receptive field without increasing parameters or reducing resolution. The atrous rate r controls the spacing between kernel weights. ASPP applies parallel atrous convolutions with different rates to capture objects at multiple scales. The encoder-decoder structure (in DeepLabv3+) first extracts features, then gradually recovers spatial details through the decoder with skip connections.
2. **Methodology:** Atrous convolution with rate r :

$$y[i] = \sum_{k=1}^K x[i + r \cdot k] \cdot w[k]$$

where x is the input, w is the filter, and y is the output.

ASPP applies convolutions at multiple rates:

$$F_{\text{ASPP}} = \text{Concat}(f_{r_1}(F), f_{r_2}(F), \dots, f_{r_n}(F), \text{GAP}(F))$$

where f_{r_i} denotes atrous convolution with rate r_i and GAP is global average pooling.

The decoder upsamples low-level features:

$$F_{\text{dec}} = \text{Upsample}(F_{\text{ASPP}}) + W_{\text{skip}} * F_{\text{low}}$$

Final prediction:

$$P = \text{softmax}(W_{\text{out}} * F_{\text{dec}})$$

3. **Input:** An RGB image of size $H \times W \times 3$, typically pre-processed with mean subtraction and normalization.

4. **Output:** A semantic segmentation map of size $H \times W$, where each pixel is assigned a class label from C possible classes.

5. **Algorithm:**

```
1   1. Extract features using backbone with atrous convolution
2   2. Apply ASPP with multiple dilation rates
3   3. Concatenate multi-scale features
4   4. Upsample in decoder with skip connections
5   5. Apply 1x1 convolution for class scores
6   6. Generate final segmentation map
7
```

2.2.5 Transformer-Based Segmentation

Vision Transformers (ViTs) adapt the self-attention mechanism from NLP to computer vision. TransUNet, introduced by Chen et al. in 2021, combines Transformers with U-Net architecture for medical image segmentation [chen2021transunet].

1. **Mechanics:** TransUNet uses a CNN to extract initial features, then patches the feature map and processes it through Transformer encoder layers. The self-attention mechanism captures long-range dependencies between image patches, overcoming the limited receptive field of CNNs. The Transformer output is reshaped and fed into a

CNN decoder similar to U-Net, with skip connections from the CNN encoder for recovering spatial details.

2. **Methodology:** The image is first processed by a CNN encoder:

$$F_{\text{CNN}} = f_{\text{CNN}}(I)$$

Features are divided into patches and flattened:

$$z_0 = [\text{Flatten}(F_1), \text{Flatten}(F_2), \dots, \text{Flatten}(F_N)] + E_{\text{pos}}$$

where E_{pos} are learnable positional embeddings.

Multi-head self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where $Q = zW_Q$, $K = zW_K$, $V = zW_V$.

Transformer layer:

$$\begin{aligned} z'_\ell &= \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1} \\ z_\ell &= \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell \end{aligned}$$

where MSA is multi-head self-attention, LN is layer normalization, and MLP is a feed-forward network.

The decoder upsamples and fuses:

$$M = f_{\text{decoder}}(\text{Reshape}(z_L), F_{\text{CNN}})$$

3. **Input:** An image of size $H \times W \times C$, divided into patches of size $P \times P$.
4. **Output:** A segmentation mask of size $H \times W \times K$, where K is the

number of semantic classes.

5. Algorithm:

- 1 1. Extract hierarchical features using CNN encoder
- 2 2. Divide feature map into patches and flatten
- 3 3. Add positional embeddings
- 4 4. Process through Transformer encoder layers
- 5 5. Reshape Transformer output to spatial dimensions
- 6 6. Upsample through CNN decoder with skip connections
- 7 7. Generate final segmentation mask
- 8

2.2.6 Recurrent Neural Networks for Segmentation

RNNs, particularly Long Short-Term Memory (LSTM) networks, have been applied to image segmentation to model spatial dependencies and context [visin2015rnn]. ReNet and related architectures process images sequentially in horizontal and vertical sweeps.

1. **Mechanics:** RNN-based segmentation treats the image as a sequence, processing it row-by-row or with overlapping patches. Four RNN layers sweep the image in different directions (left-to-right, right-to-left, top-to-bottom, bottom-to-top) to capture context from all spatial neighbors. The hidden states from all four directions are concatenated to form a rich contextual representation for each pixel. Multiple ReNet layers can be stacked to increase the receptive field.
2. **Methodology:** For a feature map $F \in \mathbb{R}^{H \times W \times C}$, four RNN sweeps compute:

$$\begin{aligned}
 h_{i,j}^{\rightarrow} &= \text{RNN}(F_{i,j}, h_{i,j-1}^{\rightarrow}) \\
 h_{i,j}^{\leftarrow} &= \text{RNN}(F_{i,j}, h_{i,j+1}^{\leftarrow}) \\
 h_{i,j}^{\downarrow} &= \text{RNN}(F_{i,j}, h_{i-1,j}^{\downarrow}) \\
 h_{i,j}^{\uparrow} &= \text{RNN}(F_{i,j}, h_{i+1,j}^{\uparrow})
 \end{aligned}$$

Hidden states are concatenated:

$$H_{i,j} = [h_{i,j}^{\rightarrow}, h_{i,j}^{\leftarrow}, h_{i,j}^{\downarrow}, h_{i,j}^{\uparrow}]$$

For segmentation, additional layers map to class predictions:

$$P_{i,j} = \text{softmax}(W \cdot H_{i,j})$$

3. **Input:** A feature map from a CNN backbone of size $H \times W \times C$, or directly an image if using raw pixels.
4. **Output:** A segmentation map of size $H \times W \times K$, where K is the number of classes.

5. **Algorithm:**

```
1   1. Extract features using CNN (if applicable)
2   2. Initialize hidden states for four RNN directions
3   3. For each layer:
4     a. Sweep left-to-right, accumulating context
5     b. Sweep right-to-left, accumulating context
6     c. Sweep top-to-bottom, accumulating context
7     d. Sweep bottom-to-top, accumulating context
8     e. Concatenate directional hidden states
9   4. Apply fully connected layer for class prediction
10  5. Output per-pixel class probabilities
11
```

references

Table 2.1: Comparison Table

Papers	Mechanics	Methodology			Performance		
		<i>Stage 1</i>	<i>Stage 2</i>	<i>Stage 3</i>	<i>Data</i>	<i>Metric</i>	<i>R</i>
copy	More table copy ^a						

^aSample of a Table footnote.

Chapter 3

Methodology

3.1 The SA-1B Data Engine — The Flywheel of Generalization

3.1.1 Paradigm Shift: From Manual Annotation to Model-in-the-Loop

The fundamental challenge in building a foundation model for segmentation is the lack of web-scale segmentation data. Unlike language or image-text pairs abundant on the internet, segmentation masks are not naturally occurring and must be manually created. Traditional approaches would require paying annotators to label billions of masks—economically infeasible and temporally prohibitive.

SAM’s solution is elegant but non-obvious: **leverage the model itself to assist, accelerate, and eventually replace human annotation.** This is the "Data Flywheel" concept—a three-stage iterative process where model improvement unlocks more efficient data collection, which in turn improves the model.

3.1.2 Stage 1: Assisted-Manual Annotation (4.3M Masks, 120K Images)

In the first stage, SAM operates as an **interactive segmentation assistant**, similar to traditional click-based segmentation tools (e.g., Grab-Cut, RITM, SimpleClick). The critical insight here is that the model’s role is to *accelerate* human labeling, not replace it.

The Workflow:

1. Professional annotators view an image in a web-based interface.
2. They click foreground/background points to indicate an object of interest.
3. SAM’s pre-computed image embedding (from MAE-ViT-H) is queried with these prompts.
4. The lightweight Prompt Encoder and Mask Decoder run in real-time ($< 50\text{ms}$), generating an initial mask prediction.
5. Annotators refine the mask using pixel-precise brush and eraser tools.
6. The final mask is saved.

Quantitative Dynamics:

- **Initial annotation time:** 34 seconds per mask (comparable to COCO annotation standards).
- **Model capacity progression:** ViT-B $\rightarrow$ ViT-H during this stage (1 retraining cycle per major improvement).
- **Average masks per image:** 20 $\rightarrow$ 44 (increased diversity as model improved).
- **Total masks collected:** 4.3M from 120K images.

Why This Matters: Unlike fully automatic approaches that can only label what the model "already knows," human annotators introduce *novel object categories, rare poses, and edge cases* that force the model to generalize. Critically, annotators were **not constrained to semantic categories**—they could label "stuff" (amorphous regions like sky, grass) and "things" (discrete objects like people, cars) uniformly, maximizing the diversity of training signals.

Efficiency Gain Analysis: The $34 \rightarrow 14$ second progression represents a **$6.5\times$ speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.

3.1.3 Stage 2: Semi-Automatic Annotation (5.9M Additional Masks, 180K Images)

In Stage 2, the goal shifts explicitly: **increase mask diversity**. The intuition is that Stage 1 naturally biases toward prominent, easy-to-label objects (due to annotators' tendency to prioritize high-salience items). Stage 2 forces the model to learn harder, less obvious objects.

The Semi-Automatic Protocol:

1. Train a generic object detector on all masks from Stage 1. This detector outputs bounding boxes regardless of semantic class (object-agnostic proposal generation).
2. SAM automatically segments high-confidence regions identified by this detector, generating candidate masks.
3. Annotators are presented with images pre-filled with these automatic masks.
4. Annotators' task: Label only the *unannotated* objects (those not covered by the automatic masks).

5. This forces annotators to focus on smaller, more occluded, or semantically ambiguous objects.

Quantitative Dynamics:

- **Annotation time (excluding automatic masks):** ~ 34 seconds per mask (unchanged; challenging objects require the same effort).
- **Average masks per image:** $44 \rightarrow 72$ (including automatic masks; diversity increased substantially).
- **New masks collected:** 5.9M, raising the cumulative total to 10.2M masks.
- **Retraining cycles:** 5 full model retraining iterations.

Why This Engineering is Critical: The semi-automatic stage is a **curriculum learning** strategy. By automatically handling easy cases and forcing annotators to label hard cases, the model learns a more balanced representation. This is empirically validated: the model’s ability to handle edge cases improved significantly from Stage 1 to Stage 2, as evidenced by the diversity metrics.

Annotation Bias Mitigation: Without Stage 2, the model would develop a strong bias toward dominant objects (people, vehicles, large structures). The semi-automatic strategy explicitly breaks this bias by requiring exhaustive annotation of all objects, regardless of prominence.

3.1.4 Stage 3: Fully Automatic Annotation (1.1B Masks, 11M Images)

This is the stage where the data flywheel reaches critical velocity. With sufficient diverse masks from Stages 1 and 2, the model is now capable enough to generate masks **without any human intervention**.

The 32×32 Grid Prompting Strategy

The core insight is deceptively simple: **systematically prompt the model with a regular spatial grid of points, covering the entire image.**

Mathematical Formulation:

- **Grid resolution:** $32 \times 32 = 1024$ grid points per image.
- Each point is passed as a prompt to the Prompt Encoder.
- For each point, SAM predicts **three masks simultaneously** (owing to its ambiguity-aware design).
- **Theoretical maximum:** $1024 \text{ points} \times 3 \text{ masks per point} = 3072 \text{ masks per image}$

Why This Grounding Works: A regular grid ensures comprehensive spatial coverage. If an object’s boundary passes through or near a grid point, the model will capture it from that point. Objects of varying sizes are covered because the grid is at a fixed, moderate resolution. This is fundamentally different from random point sampling, which can miss objects entirely.

Multi-Scale Masking and Subpart Prediction

A critical architectural feature of SAM (and one of the reasons Stage 3 is feasible) is its **ambiguity-aware decoder** that outputs multiple masks per prompt.

The Output Token Design:

- The Mask Decoder predicts three masks from a single prompt, ranked by IoU score.
- **Whole object:** The complete, semantically coherent object.

- **Part:** A prominent sub-component (e.g., a wheel on a car, a face on a person).
- **Sub-part:** Fine-grained components (e.g., an eye within a face, a spoke within a wheel).

Why Three Outputs? Empirical analysis during SAM’s development showed that image regions typically have at most three levels of semantic hierarchy (whole > part > sub-part). Predicting three masks covers all common ambiguity scenarios without excessive computation.

Mathematical Formulation: During training, SAM employs **minimum loss** over the three mask outputs:

$$\mathcal{L}_{\text{total}} = \min(\mathcal{L}_{i=1}^3) \quad \text{for masks } m_1, m_2, m_3$$

This ensures that at least one of the three masks aligns with ground truth, forcing the model to learn diverse interpretations of ambiguous prompts.

Filtering, Deduplication, and Non-Maximum Suppression

The Critical Question: If a 32×32 grid yields up to 3072 theoretical masks, why do we only get ~ 100 high-quality masks per image?

Answer: Aggressive filtering at multiple stages.

Stage 3.A: IoU Scoring and Confidence Thresholding

- For each of the three masks predicted from a grid point, SAM outputs a **predicted IoU score** (estimated mask quality).
- IoU prediction is a separate output head in the Mask Decoder: a learned estimate of the mask’s alignment with the true object boundary.
- **Threshold:** Only masks with **predicted IoU > 0.88** are retained.

- **Rationale:** High IoU threshold ensures only high-quality masks enter the deduplication pipeline.
- **Expected reduction:** From 3072 theoretical outputs, this threshold alone eliminates $\sim 80\%$ of predictions (keeping only predictions the model is highly confident about).

Stage 3.B: Stability Filtering

- A mask is **stable** if small perturbations in the model's confidence threshold (e.g., changing the foreground probability threshold from 0.5 to $0.5 \pm \epsilon$) produce similar masks.
- **Operationally:** Threshold the mask's probability map at both 0.5 and 0.5, and compute the IoU between the two resulting binary masks.
- If this "stability IoU" is high (> 0.95), the mask is kept; otherwise, it is discarded.
- **Why This Matters:** Masks that are brittle (sensitive to small threshold changes) are often segmentation errors or include spurious boundaries. Stability filtering removes low-confidence predictions and keeps robust ones.
- **Further reduction:** $\sim 50\%$ of remaining masks fail stability checks.

Stage 3.C: Non-Maximum Suppression (NMS) for Mask Deduplication After IoU and stability filtering, many remaining masks are **duplicates or near-duplicates** (same object segmented from slightly different grid points, or different granularities of the same object).

Traditional NMS for Bounding Boxes: Classical NMS (used in object detection) operates on bounding boxes and IoU: 1. Sort boxes by confidence score. 2. Select the highest-confidence box. 3. Remove all boxes with $\text{IoU} > \text{threshold}$ to this box. 4. Repeat on remaining boxes.

Mask-Based NMS in SAM: The SAM paper adapts NMS for mask deduplication: 1. Sort masks by predicted IoU score. 2. Select the highest-scoring mask m_1 . 3. For all remaining masks m_i :

- Compute $\text{IoU}(m_1, m_i) = \frac{|m_1 \cap m_i|}{|m_1 \cup m_i|}$ (Jaccard index).
- If $\text{IoU}(m_1, m_i) > \text{thresh}_{\text{IoU}}$ (typically 0.7), discard m_i (it is a near-duplicate).
- If $\text{IoU}(m_1, m_i) \leq \text{thresh}_{\text{IoU}}$, retain m_i (it is a distinct object).

4. Repeat on remaining unprocessed masks.

Result: NMS eliminates the redundant foreground predictions while preserving distinct objects, even when they overlap partially.

Scale Reduction Summary:

- **Start:** 3072 theoretical masks (32×32 grid $\times$ 3 outputs per grid cell).
- **After IoU filtering (> 0.88):** ~ 600 masks (20% remain).
- **After stability filtering:** ~ 300 masks (50% of IoU-filtered).
- **After NMS ($\text{thresh_IoU} = 0.7$):** ~ 100 masks (final).

Final Calculation:

$$\text{Total SA-1B Masks} = 11\text{M images} \times 100 \text{ masks/image} = 1.1 \text{ B masks}$$

Multi-Scale Processing: Zoomed-in Image Crops

To further improve small object segmentation (which is notoriously challenging), SAM processes **multiple overlapping zoomed-in crops** of each image.

The Zooming Strategy:

- Original image is processed at full resolution (1500 pixels shortest side after downsampling).
- Additionally, crop and zoom into regions systematically (e.g., $2\times$, $4\times$ magnification).
- The 32×32 grid prompting is re-applied to these zoomed crops.
- Additional fine-grained masks are generated for small objects that might be missed in the full-resolution pass.
- These zoomed-crop masks are appended to the final set (before final NMS).

Why This Is Crucial: Small objects (relative size $< 1\%$ of image) have very few pixels. A standard NMS threshold might inadvertently merge them with larger neighbors. By processing zoomed crops, SAM explicitly allocates model capacity to these hard-to-segment regions.

Chapter 4

Cài đặt và thử nghiệm

Chapter 5

Kết luận và hướng phát triển

5.1 Conclusion

5.2 Discussion

5.2.1 Limitations

5.2.2 Future Work

“ [7]